

DPNet: Doppler LiDAR Motion Planning for Highly-Dynamic Environments

Wei Zuo¹, Zeyi Ren¹, Chengyang Li¹, Yikun Wang¹, Mingle Zhao³, Shuai Wang^{2,†},
Wei Sui⁴, Fei Gao⁵, Yik-Chung Wu^{1,†}, and Chengzhong Xu³.

Abstract—Existing motion planning methods often struggle with rapid-motion obstacles due to an insufficient understanding of environmental changes. To address this, we propose integrating motion planners with Doppler LiDARs, which provide not only ranging measurements but also instantaneous point velocities. However, this integration is nontrivial due to the requirements of high accuracy and high frequency. To this end, we introduce Doppler Planning Network (DPNet), which tracks and reacts to rapid obstacles via Doppler model-based learning. We first propose a Doppler Kalman neural network (D-KalmanNet) to track obstacle states under a partially observable Gaussian state space model. We then leverage the predicted motions of obstacles to construct a Doppler-tuned model predictive control (DT-MPC) framework for ego-motion planning, enabling runtime auto-tuning of controller parameters. These two modules allow DPNet to learn fast environmental changes from minimal data while remaining lightweight, achieving high frequency and high accuracy in both tracking and planning. Experiments on high-fidelity simulator and real-world datasets demonstrate the superiority of DPNet over extensive benchmark schemes.

Index Terms—Integrated planning and learning, collision avoidance, Doppler motion planning

I. INTRODUCTION

REAL-TIME robot motion planning in highly-dynamic environments is crucial for a broad range of applications, including emergency rescue and autonomous driving [1]. Such scenarios require accurate knowledge of environmental dynamics, which can be leveraged by the motion controller to generate sequences of collision-free actions under kinematic constraints. Existing motion planning methods heavily rely on light detection and ranging (LiDAR) sensors to capture the current positions of the obstacles from the point cloud [2], and predict obstacles future trajectories via learning-based [3], physics-based [4], or physics-informed learning [5], [6] methods. Nonetheless, these approaches struggle to handle rapid-

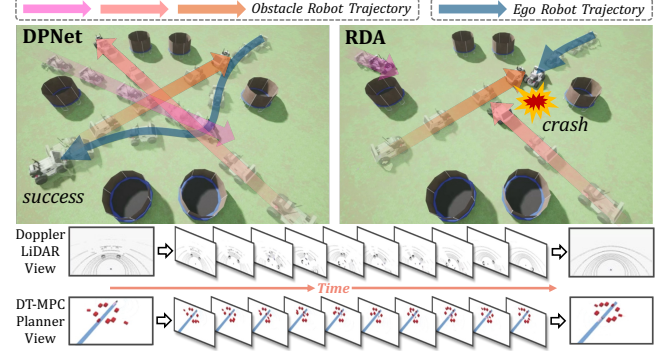


Fig. 1: Comparison between DPNet and RDA [8]. DPNet can understand and traverse highly-dynamic environments safely.

motion obstacles due to their limited ability to understand real-time environmental changes, such as instantaneous velocities.

Recently, however, the situation has changed thanks to the remarkable milestone of the emerging Doppler LiDAR sensors, which not only offer traditional point ranging data but also point Doppler velocity [7]. The additional dimension explicitly provides instantaneous obstacle motion information, opening up new opportunities for motion planning in rapidly changing scenarios. Yet, how to integrate Doppler LiDAR into motion planners within a unified solution framework remains an open challenge. This is nontrivial since the solution should be high-performance, real-time, safety-assured, and work properly with limited onboard computational resources.

Given certain predictions of obstacle future states, the next step is motion control. Conventional motion controllers, e.g., model predictive control (MPC), use the predicted states to construct consecutive problems along the horizon [9]. However, they may suffer from suboptimality if the prior knowledge about the cost function or the dynamics model is imperfect. To address this, emerging auto-tuning MPC enables parameter learning from closed-loop execution data [10]. Nevertheless, when applied to collision avoidance, these tune-from-execution methods typically require actual collision events to trigger the process. To broaden their applications, how to leverage Doppler information to trigger runtime tuning without relying on actual events becomes an important issue.

To fill these research gaps, this paper proposes Doppler planning network (DPNet). As shown in Fig. 1, DPNet can traverse highly-dynamic areas by leveraging Doppler LiDAR and introducing two key algorithmic innovations for obstacle motion learning and runtime controller tuning. The first innovation is a Doppler Kalman neural network (D-KalmanNet) which achieves real-time obstacle future state prediction under a partially observable Gaussian state space

Project Page: <https://github.com/UUwei-zuo/DPNet>

Manuscript received: November 27, 2025 ; Revised: February 28, 2026; Accepted: April 9, 2026. This paper was recommended for publication by Editor Ashis Banerjee upon evaluation of the Associate Editor and Reviewers' comments. This work was supported by National Natural Science Foundation of China (Grant No. 62371444), TUBITAK-CAS Bilateral Cooperation Program (Grant No. 321GJHZ2025118MI), MOST and The Science and Technology Development Fund (FDCT) of Macau SAR (File no. 0074/2025/AMJ), Shenzhen Science and Technology Program (Grant No. RCYX20231211090206005, JCYJ20241202124934046), and SIAT Original Exploration Program.

¹ Wei Zuo, Zeyi Ren, Chengyang Li, Yikun Wang, and Yik-Chung Wu are with The University of Hong Kong, Hong Kong ({zuowei, renzeyi, ykwang, ycwu}@eee.hku.hk and chengyangli@connect.hku.hk).

² Shuai Wang is with Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, China (s.wang@siat.ac.cn).

³ Mingle Zhao and Chengzhong Xu are with University of Macau, Macau (zhao.mingle@connect.um.edu.mo and czxu@um.edu.mo).

⁴ Wei Sui is with D-Robotics, China (wei.sui@d-robotics.cc).

⁵ Fei Gao is with Zhejiang University, China (fgaoaa@zju.edu.cn).

[†] Corresponding authors: Shuai Wang and Yik-Chung Wu.

(GSS) model. The D-KalmanNet incorporates the structural GSS model with a recurrent neural network (RNN) and sequences of Doppler velocity measurements, ensuring a very high tracking frequency and a lightweight model size. The second innovation is a Doppler-tuned model predictive control (DT-MPC) framework that performs Doppler-inferred collision check to trigger runtime parameter tuning from imagined collisions instead of actual ones. For computational efficiency, a heuristic update strategy is designed and an alternating direction method of multipliers (ADMM) algorithm is used for controller execution. Built upon these innovations, DPNet can understand and cope with rapid environmental changes using minimal data.

To evaluate DPNet with D-KalmanNet and DT-MPC algorithms, extensive robot operation system (ROS) experiments are performed. We show that DPNet outperforms state-of-the-art planners including RDA [8], MPC-D-CBF [11], and OBCA [12] on diverse metrics in Carla simulator [13]. Ablation study and sensitivity analysis confirm DT-MPC's effectiveness and robustness in triggering parameter updates without actual events, especially in cluttered environments. Moreover, evaluations on real-world Doppler LiDAR dataset Aevascene [14] demonstrate that D-KalmanNet achieves lowest tracking errors over existing tracking methods including Kalman filter [11] and KalmanNet [5] across wide execution frequency range (i.e., 1–10 Hz), different horizon lengths (1–10), and various road scenarios (i.e., *Highway* and *City*). It is also found that D-KalmanNet can be executed at 15–100 Hz on a low-cost NVIDIA Jetson Orin NX 16 GB when tracking up to 10 objects simultaneously. To the best of our knowledge, *this is the first attempt to integrate Doppler LiDAR with motion planning*. Our contributions are summarized as follows:

- We introduce Doppler LiDAR to support motion planner designs, resulting in a novel DPNet framework. By incorporating Doppler velocities for obstacle prediction, DPNet can proactively avoid highly-dynamic obstacles.
- We propose D-KalmanNet for obstacle state tracking and prediction by model-based learning, achieving higher accuracy than existing methods while maintaining low computational cost and high frequency.
- We propose DT-MPC for runtime controller tuning by triggering updates from Doppler-inferred collisions.
- We implement the complete DPNet framework for evaluation. Extensive results show significant improvements over existing schemes in terms of diverse metrics.

II. RELATED WORK

Doppler LiDAR. In contrast to traditional LiDAR [2], Doppler LiDAR reshapes the landscape of robotic solutions by providing Doppler velocity measurements. It has been used in object detection, state estimation, and simultaneous localization and mapping [15]–[21]. In terms of evaluation, Carla [13] provides realistic Doppler LiDAR simulations [7], [22], and AevaScenes [14] dataset provides on-road Doppler LiDAR recordings. However, all existing research focuses on open-loop perception usages, and integrating Doppler LiDAR into closed-loop motion planning remains unexplored.

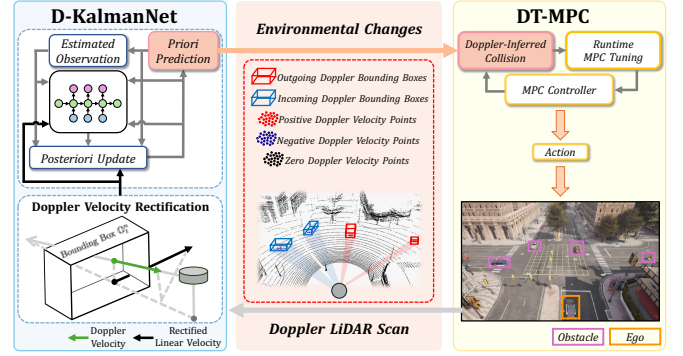


Fig. 2: The proposed DPNet system, which consists of D-KalmanNet and DT-MPC modules.

Obstacle Tracking. Incorporating forecasts of obstacle future trajectories can significantly enhance planning performance. Currently, obstacle tracking can be divided into learning-based, physics-based, and hybrid methods. Learning-based methods require large datasets and substantial computational resources for training [23], [24]. Physics-based methods (e.g., Kalman filter [4]) capitalize on prior knowledge for high computational efficiency, while relying on predefined kinematic models that introduce inevitable bias and limit the ability to handle complex motions, especially when the state transition model is partially observed. Lastly, hybrid methods combine strengths of both categories, resulting in efficient and adaptable physics-informed learning solutions. For example, the recent KalmanNet [5] mitigates the drawbacks of Kalman filters by learning the Kalman gain from real data, so as to maintain both high inference accuracy and lightweight model size. This work extends physics-informed learning to Doppler LiDAR, which has not been studied yet.

Motion Planning. Many modern motion planning systems (e.g., robot, autonomous vehicle) are based on MPC, which generates optimal control outputs in response to environmental inputs within a receding horizon window [25]. By incorporating obstacle tracking, MPC can construct time-varying constraints and handle dynamic environments [9]. In general, the MPC performance depends on the problem modeling. To find the best setting, emerging auto-tuning MPC [10] uses closed-loop feedback to optimize controller parameters. However, it typically involves time-consuming data collections or requires actual events to trigger. In contrast, DT-MPC achieves runtime MPC tuning by inferring collisions from instantaneous Doppler measurements and obstacle state tracking, boosting real-time robustness for handling rapid-motion obstacles.

III. PROBLEM FORMULATION

We consider an MPC motion planner operating among N dynamic obstacles $\mathcal{N} = \{1, \dots, N\}$. At time t , the MPC horizon is represented by a finite sequence of H discretized future steps $\mathcal{H}_t \triangleq \{t, \dots, t + H - 1\}$, where Δt is the time length between two states. Let $\mathbf{s}_h = (x_h, y_h, \theta_h)$ be the robot state, where (x_h, y_h) and θ_h are position and orientation at the bottom center of the corresponding bounding box $\mathbb{G}_h(\mathbf{s}_h)$, respectively. Let $\mathbf{w}_h = (\mathbf{v}_h, \psi_h)$ be the robot action, where $\mathbf{v}_h$ and ψ_h are linear and angular velocity, respectively. By

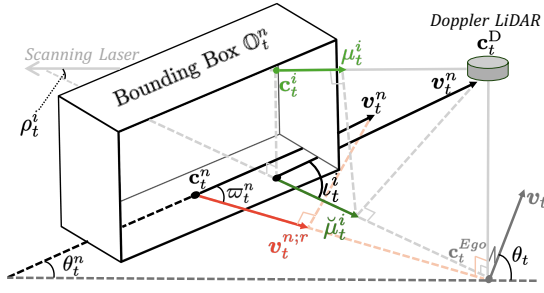


Fig. 3: Doppler velocity rectification.

iterating over $\mathcal{H}_t$, the MPC system finds an optimal state sequence $\mathcal{S}_t = \{s_{t+1}, s_{t+2}, \dots, s_{t+H}\}$ via action sequence $\mathcal{W}_t = \{w_t, w_{t+1}, \dots, w_{t+H-1}\}$, where $\{\mathcal{W}_t, \mathcal{S}_t\} \in \mathcal{F}_t$, with $\mathcal{F}_t$ being the kinematic constraint detailed in Section IV. The problem is thus formulated as

$$P_t : \min_{\{\mathcal{W}_t, \mathcal{S}_t\} \in \mathcal{F}_t} C_t(\mathcal{S}_t) \quad (1a)$$

$$\text{s.t.} \quad \text{dist}(\mathbb{G}_{h+1}, \mathbb{O}_{h+1}^n) \geq d_{\text{safe}}, \forall h \in \mathcal{H}_t, n \in \mathcal{N}, \quad (1b)$$

where $C_t(\mathcal{S}_t)$ is the utility function, d_{safe} is the safety distance, and $(\mathbb{G}_{h+1}, \mathbb{O}_{h+1}^n)$ are bounding boxes of ego-robot and the n -th obstacle, respectively. The minimum distance function between two bounding boxes is defined as $\text{dist}(\mathbb{P}, \mathbb{Q}) = \min\{\|\mathbf{p} - \mathbf{q}\|_2 \mid \mathbf{p} \in \mathbb{P}, \mathbf{q} \in \mathbb{Q}\}$. Considering a reference path tracking task with target waypoints $\{s_{h+1}^* \mid h \in \mathcal{H}_t\}$, we formulate $C_t(\mathcal{S}_t) = \sum_{h \in \mathcal{H}_t} \|s_{h+1} - s_{h+1}^*\|_2^2$ [12].

Problem P_t is nontrivial since estimating $\mathbb{O}_{h+1}^n$ results in an inevitably biased $\hat{\mathbb{O}}_{h+1|t}^n$, which can significantly mismatch the actual $\mathbb{O}_{h+1}^n$ if obstacle n has rapid motions. This error would further propagate to motion planning since $\text{dist}(\mathbb{G}_{h+1}, \hat{\mathbb{O}}_{h+1|t}^n) \neq \text{dist}(\mathbb{G}_{h+1}, \mathbb{O}_{h+1}^n)$.

IV. DOPPLER PLANNING NETWORK

To solve P_t among rapid-motion obstacles, we propose D-KalmanNet for reducing the uncertainty of $\hat{\mathbb{O}}_{h+1|t}^n$ and DT-MPC for robustifying the solution by real-time tuning d_{safe} . We then integrate them into a unified solution, DPNet, whose overall architecture is shown in Fig. 2. It takes the Doppler LiDAR point cloud $\mathcal{P}_t$ as input and outputs the actions $\mathcal{W}_t$.

A. Doppler KalmanNet for Tracking

The ego robot uses a Doppler LiDAR to scan the surrounding environment and obtain Doppler point cloud $\mathcal{P}_t = \{\mathbf{c}_t^i, \|\boldsymbol{\mu}_t^i\|_2\}_{i=1}^I$, where I is the number of points in the frame, $\mathbf{c}_t^i \in \mathbb{R}^3$ and $\|\boldsymbol{\mu}_t^i\|_2 \in \mathbb{R}$ are the spatial coordinate and the scalar Doppler velocity of the i -th point, respectively. Accordingly, we define the Doppler-augmented state $\mathbf{x}_t^n$ as

$$\mathbf{x}_t^n = [x_t^n, \cos \theta_t^n v_t^n, \cos \theta_t^n a_t^n, y_t^n, \sin \theta_t^n v_t^n, \sin \theta_t^n a_t^n]^\top, \quad (2)$$

where (x_t^n, y_t^n) , θ_t^n , v_t^n and a_t^n represent the position, orientation, linear velocity and acceleration at the center, respectively. Each $\mathbf{x}_t^n$ can be used to generate a $\mathbb{O}_t^n$. Hence, we only need to estimate a sequence of $\{\hat{\mathbf{x}}_{t+1|t}^n, \hat{\mathbf{x}}_{t+2|t}^n, \dots, \hat{\mathbf{x}}_{t+H|t}^n\}$ from $\mathcal{P}_t$. We can fuse $\mathcal{P}_t$ with a state transition model to achieve this goal, but two challenges exist. First, according to Doppler LiDAR manufacturers [14], real-world velocity measurements typically involve noises with standard deviation of 0.1 m/s. Second, built upon past trajectories, the transition model may

Algorithm 1 D-KalmanNet

Input: $\{\mathbf{y}_0^n, \mathbf{y}_1^n, \dots\}_{n=1}^N$
Output: $\{\hat{\mathbb{O}}_0^n, \hat{\mathbb{O}}_1^n, \dots\}_{n=1}^N$

- 1: Initialize $t = 0$
- 2: **while** *True* **do**
- 3: **for** $n \in \mathcal{N}$ **do**
- 4: Group $\mathcal{P}_t^n$ to obtain $\mathbf{y}_t^n$
- 5: **if** $t = 0$ **then** // Initial Guess
- 6: $\mathbf{x}_0^n \leftarrow \mathbf{y}_0^n, \hat{\mathbf{x}}_{1|0}^n \leftarrow \mathbf{T} \cdot \mathbf{x}_0^n, \hat{\mathbf{y}}_{1|0}^n \leftarrow \mathbf{U} \cdot \hat{\mathbf{x}}_{1|0}^n$
- 7: **else** // Kalman Learning
- 8: $\mathcal{K}_t^n \leftarrow \text{RNN}(\mathbf{x}_{t-1}^n, \hat{\mathbf{x}}_{t|t-1}^n, \hat{\mathbf{y}}_{t|t-1}^n, \mathbf{y}_t^n)$
- 9: $\mathbf{x}_t^n \leftarrow \hat{\mathbf{x}}_{t|t-1}^n + \mathcal{K}_t^n \cdot (\mathbf{y}_t^n - \hat{\mathbf{y}}_{t|t-1}^n)$
- 10: $\hat{\mathbf{x}}_{h+1|t}^n \leftarrow \mathbf{T}^{h-t+1} \cdot \mathbf{x}_t^n, \forall h \in \mathcal{H}_t$
- 11: $\hat{\mathbf{y}}_{t+1|t}^n \leftarrow \mathbf{U} \cdot \hat{\mathbf{x}}_{t+1|t}^n$
- 12: **end if**
- 13: return $\hat{\mathbb{O}}_t^n = \{\hat{\mathbb{O}}_{h+1|t}^n(\hat{\mathbf{x}}_{h+1|t}^n)\}_{\forall h \in \mathcal{H}_t}$
- 14: **end for**
- 15: $t \leftarrow t + 1$
- 16: **end while**

mismatch the actual obstacle motions in the future [5]. Our D-KalmanNet can simultaneously tackle both challenges.

1) *Doppler Velocity Rectification:* We propose Doppler velocity rectification shown in Fig. 3 to address the first challenge. Our observation is that any measurement point on a common obstacle with a rigid-body motion would share the same linear velocity. As such, it is possible to aggregate velocities across dense points for mitigating the Doppler LiDAR noise. Since different lasers hit different points on obstacle surfaces and reflect different radial velocities, we first unify the measurements to the same vector direction. Specifically, let $\mathbf{c}_t^n, \mathbf{c}_t^{\text{Ego}} \in \mathbb{R}^3$ represent positions of obstacle bottom center and ego robot bottom center, respectively. The Doppler LiDAR mounted at $\mathbf{c}_t^{\text{D}} \in \mathbb{R}^3$ emits its i -th scanning laser with a scanning elevation angle ρ_t^i , hitting and reflecting from point $\mathbf{c}_t^i$. The Doppler velocity projection $\tilde{\boldsymbol{\mu}}_t^i$ on the 2D plane is derived as:

$$\tilde{\boldsymbol{\mu}}_t^i = \frac{\|\boldsymbol{\mu}_t^i\|_2}{\cos \rho_t^i} \times \frac{(\mathbf{I} - \mathbf{e}\mathbf{e}^\top)(\mathbf{c}_t^{\text{D}} - \mathbf{c}_t^i)}{\|(\mathbf{I} - \mathbf{e}\mathbf{e}^\top)(\mathbf{c}_t^{\text{D}} - \mathbf{c}_t^i)\|_2}, \quad (3)$$

where $\mathbf{e} = [0 \ 0 \ 1]^\top$ and $\mathbf{I} \in \mathbb{R}^{3 \times 3}$ is an identity matrix.

With the projection, we then group Doppler points inside the obstacle bounding box as $\mathcal{P}_t^n = \{\{\mathbf{c}_t^i, \|\boldsymbol{\mu}_t^i\|_2\} \mid \mathbf{c}_t^i \subseteq \mathbb{O}_t^n\} = \{p_t^{i_n}\}_{i_n=1}^{I_n}$, where $p_t^{i_n}$ is the i_n -th point of group $\mathcal{P}_t^n$, $\subseteq$ is an operator that indicates being within the spatial boundary. Denote ϖ_t^n as the angle between $\mathbf{v}_t^n$ and center radial velocity $\mathbf{v}_{t;r}^n$, and ι_t^i as the angle between $\mathbf{v}_t^n$ and Doppler velocity projection $\tilde{\boldsymbol{\mu}}_t^i$. Based on rigid motion consistency $\mathbf{v}_{t;r}^n \cos \varpi_t^n = \tilde{\boldsymbol{\mu}}_t^i \cos \iota_t^i$, the fused linear velocity v_t^n is given by

$$v_t^n = \frac{1}{|I_n|} \sum_{i_n=1}^{I_n} \frac{1}{\cos \iota_t^{i_n}} \tilde{\boldsymbol{\mu}}_t^{i_n}. \quad (4)$$

2) *Kalman Gain Learning:* We propose Kalman gain learning to address the second challenge. We set up a partially observable GSS model for the dynamics, whose state transition model is $\mathbf{T}$ (assuming a constant acceleration) and state

Algorithm 2 Doppler Collision Check-and-Tune

```

1: Initialize  $\mathcal{S}_{t-1} = \emptyset$ 
2: while True do
3:    $\varrho_n \leftarrow \infty$  for  $n \in \mathcal{N}$ ,  $\kappa \leftarrow \kappa^{\text{init}}$  // New Iteration
4:   Obtain  $\hat{\mathcal{O}}_t^n$  from the D-KalmanNet
5:   for  $h \in \mathcal{H}_{t-1}^-$  do
6:     for  $n \in \mathcal{N}$  do
7:       if  $\varrho_n = \infty$  and  $\text{dist}(\mathbb{G}_{h+1}^\circ, \hat{\mathcal{O}}_{h+1|t}^n) \leq d_0$  then
8:          $\varrho_n \leftarrow \kappa$  // Assign Collision Priority
9:       end if
10:    end for
11:    Update  $\kappa \leftarrow \kappa + \Delta\kappa$  // Update Accumulator
12:  end for
13:   $t \leftarrow t + 1$ 
14: end while

```

observation model is $\mathbf{U}$ (involving both position and velocity). Then we have the following state transition

$$\hat{\mathbf{x}}_{h+1|h}^n = \mathbf{T}\mathbf{x}_h^n, \hat{\mathbf{y}}_{h+1|h}^n = \mathbf{U}\hat{\mathbf{x}}_{h+1|h}^n, \quad (5)$$

where $\hat{\mathbf{x}}_{h+1|h}^n$ is the priori state prediction and $\hat{\mathbf{y}}_{h+1|h}^n$ is the estimated state observation using forward propagation. After Δt , new scan $\mathcal{P}_{h+1}$ arrives, providing the actual $\mathbf{y}_{h+1}^n$ by (2) and (4). We then update the posteriori state prediction by

$$\mathbf{x}_{h+1}^n = \hat{\mathbf{x}}_{h+1|h}^n + \mathcal{K}_{h+1}^n \cdot (\mathbf{y}_{h+1}^n - \hat{\mathbf{y}}_{h+1|h}^n), \quad (6)$$

where $\mathcal{K}_{h+1}^n$ is the learned Kalman gain:

$$\mathcal{K}_{h+1}^n = \text{RNN}(\mathbf{x}_h^n, \hat{\mathbf{x}}_{h+1|h}^n, \hat{\mathbf{y}}_{h+1|h}^n, \mathbf{y}_{h+1}^n). \quad (7)$$

We adopt an RNN [5] to learn a robust and adaptive $\mathcal{K}_{h+1}^n$ in (7) for posteriori update, thereby alleviating the noises originated from either mismatched $\mathbf{T}$ or inaccurate $\mathbf{y}_{h+1|h}^n$. The entire procedure is summarized in Algorithm 1, where $\hat{\mathcal{O}}_t^n = \{\hat{\mathcal{O}}_{h+1|t}^n(\hat{\mathbf{x}}_{h+1|t}^n)\}_{\forall h \in \mathcal{H}_t}$ represents bounding box predictions over the planning horizon.

B. Doppler-Tuned MPC for Planning

A fixed d_{safe} in $\mathbf{P}_t$ often leads to poor performance or problem infeasibility [10]. This can be addressed by MPC auto-tuning [26], which transforms d_{safe} into a learnable parameter vector $\phi = [\phi_{t+1,1}, \dots, \phi_{t+H,N}]^T$ and reformulates $\mathbf{P}_t$ into the following parameter optimization problem:

$$\mathbf{Q}_t : \min_{\{\mathcal{W}_t, \mathcal{S}_t\} \in \mathcal{F}_t, \phi \in \Phi} C_t(\mathcal{S}_t) + \gamma \mathcal{L}(\phi, \{\hat{\mathcal{O}}_t^n\}), \quad (8a)$$

where $\mathcal{L}$ is the closed-loop collision cost function, γ is the collision penalty coefficient and Φ defines the upper and lower bounds of ϕ . One example of $\mathcal{L}$ is the quadratic loss of the collision penalty [26], where

$$\mathcal{L}(\phi, \{\hat{\mathcal{O}}_t^n\}) = \sum_{h \in \mathcal{H}_t} \sum_{n \in \mathcal{N}} \left\| \min \left(\text{dist}(\mathbb{G}_{h+1}^\circ, \hat{\mathcal{O}}_{h+1|t}^n) - \phi_{h+1,n}, 0 \right) \right\|^2. \quad (9)$$

To minimize $\mathcal{L}$, DT-MPC adopts a heuristic rule [27] to trigger parameter updates by using $\hat{\mathcal{O}}_t^n$ for Doppler-inferred collision check. This allows exploring whether real-time Doppler measurements can facilitate tuning even with simple heuristics (see Section V-D for its limitation).

1) *Doppler-Inferred Collision Check*: In an MPC system, solving $\mathbf{Q}_t$ produces complete $\mathcal{W}_t$ and $\mathcal{S}_t$ along $\mathcal{H}_t$, while only the first action $\mathbf{w}_t$ is executed, which ends up with state $\mathbf{s}_{t+1}$. This leaves the potential state sequence $\mathcal{S}_t^\circ = \{\mathbf{s}_h^\circ\}_{h=t+2}^{t+H}$ as a natural surrogate of ego trajectory prediction. Hence, before solving $\mathbf{Q}_t$, we perform parameter tuning by assessing historical potential state sequence $\mathcal{S}_{t-1}^\circ$ from the solution of $\mathbf{Q}_{t-1}$. Specifically, $\mathcal{S}_{t-1}^\circ$ corresponds to the ego bounding box prediction $\{\mathbb{G}_{h+1}^\circ(\mathbf{s}_{h+1}^\circ)\}_{\forall h \in \mathcal{H}_{t-1}^-}$, where $\mathcal{H}_{t-1}^- = \mathcal{H}_{t-1} \setminus \{t-1\}$ denotes the truncated historical horizon. Inspired by [28], we then check potential collisions between $\{\hat{\mathcal{O}}_{h+1|t}^n(\hat{\mathbf{x}}_{h+1|t}^n)\}$ and $\{\mathbb{G}_{h+1}^\circ(\mathbf{s}_{h+1}^\circ)\}$ along $\mathcal{H}_{t-1}^-$, using the following criteria:

$$\text{dist}(\mathbb{G}_{h+1}^\circ, \hat{\mathcal{O}}_{h+1|t}^n) \leq d_0, \forall h \in \mathcal{H}_{t-1}^-, n \in \mathcal{N}, \quad (10)$$

where d_0 is the distance threshold of Doppler-inferred collision. Note that the last element of $\hat{\mathcal{O}}_t^n$ is excluded from the check to match the length of $\mathcal{H}_{t-1}^-$.

2) *Run-Time Auto-Tuning*: If such Doppler-inferred collision (10) exists for any obstacle, ϕ for $\mathbf{Q}_t$ is updated to alleviate potential future risks. We first decompose $\phi_{h,n}$ into a constant component and an adaptive component as $\phi_{h,n} = d_1 + \tau(n, h)d_2$, where d_1 is the compulsory minimum distance and $\tau(n, h)d_2$ is the Doppler adaptive distance. This converts tuning $\phi_{h,n}$ to tuning the factor $\tau(n, h)$. We further decompose $\tau(n, h) = \tau_1(n)\tau_2(h)$, where $\tau_1(n) \in (0, 1)$ is the spatial factor and $\tau_2(h) \in (0, 1)$ is the temporal factor. First, the spatial factor $\tau_1(n)$ measures the imminence of the Doppler-inferred collision by

$$\tau_1(n) = \max(1 - \alpha(\varrho_n - \kappa^{\text{init}}), \tau_{1;\min}), \quad (11)$$

where $\tau_{1;\min}$ is the lower bound, $\alpha \in (0, 1)$ is the decay rate, and ϱ_n is the collision priority assigned chronologically. A larger α leads to a higher focus on imminent collisions. Algorithm 2 details how ϱ_n is assigned using the accumulator κ , which starts with κ^{init} and increments by $\Delta\kappa$ along the horizon. Second, the temporal factor $\tau_2(h)$ accounts for the inherently increasing uncertainty of farther predictions:

$$\tau_2(h) = \max(1 - \beta(h - t), \tau_{2;\min}), \quad (12)$$

where $\tau_{2;\min}$ is the lower bound and $\beta \in (0, 1)$ is the horizon decay rate. A larger β imposes stricter constraints on nearby predictions while relaxing constraints for distant ones. Table II in Section V provides a sensitivity analysis of (α, β) .

3) *Solution by ADMM*: With the Doppler-tuned $\hat{\phi}$ from DT-MPC (Algorithm 2), $\mathbf{Q}_t$ is explicitly written as

$$\mathbf{R}_t : \min_{\mathcal{W}_t, \mathcal{S}_t} C_t(\mathcal{S}_t) + \gamma \mathcal{L}(\hat{\phi}, \{\hat{\mathcal{O}}_t^n\}), \quad (13a)$$

$$\text{s.t.} \quad \mathbf{s}_{h+1} = \mathbf{s}_h + f(\mathbf{s}_h, \mathbf{w}_h)\Delta t, \forall h, \quad (13b)$$

$$\mathbf{w}_{\min} \preceq \mathbf{w}_h \preceq \mathbf{w}_{\max}, \forall h, \quad (13c)$$

where (13b) represents the state evolution ($f(\cdot)$ defines vehicle dynamics) and (13c) represents the physical constraint ($\mathbf{w}_{\min}, \mathbf{w}_{\max}$ are control limits), i.e., the kinematic set $\mathcal{F}_t$ explicitly becomes $\mathcal{F}_t = \{(13b), (13c)\}$ [12]. To tackle the only nonconvex part $\mathcal{L}(\hat{\phi}, \{\hat{\mathcal{O}}_t^n\})$ in $\mathbf{R}_t$, we equivalently transform $\mathcal{L}$ into a biconvex form using Lagrange duality [8], [26], and then solve the resultant biconvex version by ADMM. This ADMM will converge to a stationary point of $\mathbf{R}_t$.

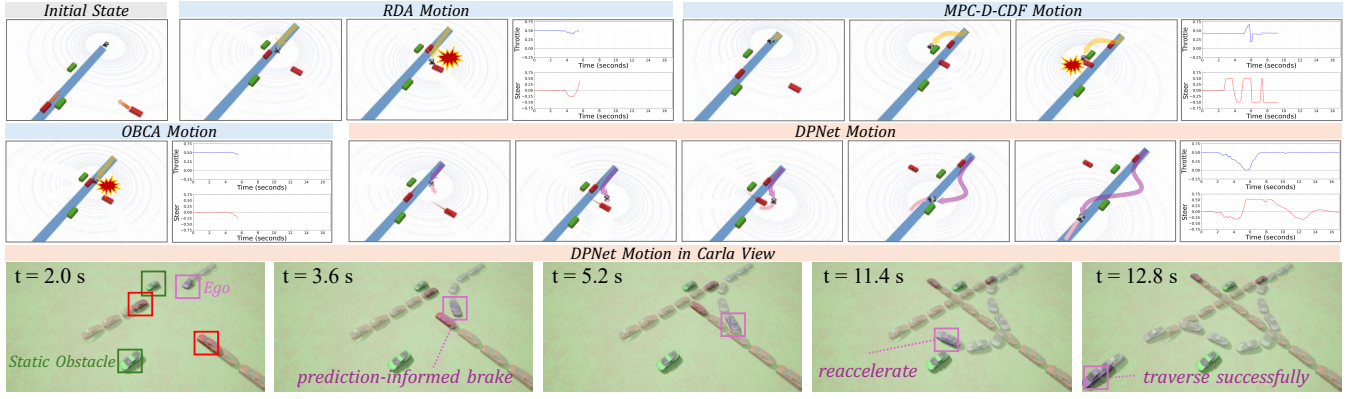


Fig. 4: Qualitative analysis of robot motions (in ROS-RViz and Carla [13] views) and corresponding control commands. Static obstacles are marked as green boxes, while dynamic ones are red boxes with arrows indicating their moving directions. The blue line is the path connecting start and goal points. The dots in front of dynamic obstacles represent motion predictions.

V. EXPERIMENTS

We implement DPNet in ROS Noetic on Linux system, using Carla simulator with Doppler LiDAR integration [22]. We compare DPNet with state-of-the-art methods and provide both qualitative and quantitative results in highly-dynamic environments randomized by DynaBARN [29] benchmark, along with ablation study and sensitivity analysis for DT-MPC. We also evaluate D-KalmanNet against existing obstacle tracking methods in highly-dynamic environments, and assess its efficiency on resource-constrained platform NVIDIA Jetson Orin NX. Ground truths of obstacle bounding boxes are used to group Doppler LiDAR points.

We train D-KalmanNet for 2000 epochs on the real-world Doppler LiDAR dataset AevaScenes [14] comprising 100 sequences (50 in *City*, 50 in *Highway*). Each sequence contains 100 frames recorded at 10 Hz with ground-truth vehicle linear velocities and bounding box poses. Training uses 80 randomly selected sequences (40 per scenario), and the remaining 20 sequences serve as the evaluation set in Section V-B.

A. End-to-End Evaluation in Carla

We compare the following methods¹ for end-to-end evaluations in Carla-ROS simulation: (1) **DPNet**: our proposed method; (2) **MPC-D-CBF** [11]: an MPC-based planner that predicts obstacle motion using LiDAR and Kalman filter, constructing D-CBFs online for navigation among moving obstacles; (3) **RDA** [8]: an MPC-based planner that uses parallelizable dual ADMM to accelerate collision avoidance; (4) **OBCA** [12]: an MPC-based planner with optimization-based collision avoidance for convex obstacles; (5) **DPNet (Ablation)**: our proposed method where DT-MPC is disabled. All methods are set up with $H = 15$ and $\Delta t = 0.1$ s. DPNet’s specific parameters are set as follows: $d_0 = 0.5$ m, $d_1 = 0.1$ m, $d_2 = 0.4$ m, $\alpha = 0.2$, $\beta = 0.05$, $\tau_{1;\min} = 0.3$, $\tau_{2;\min} = 0.3$, $\kappa^{\text{init}} = 1.0$, and $\Delta\kappa = 1.0$. Ego robot and obstacles are modeled as car-like wheel robots.

Qualitative Analysis. As shown in Fig. 4, this scenario consists of two static obstacles and two rapid-motion obstacles (at 5 m/s) with frontal and lateral directions. These obstacles

will not avoid the ego robot. The ego robot needs to traverse the area without any collisions. It can be seen that our DPNet succeeds to pass through the area with agile motions. Particularly, at $t = 2$ s, if velocity information is ignored, directly passing between the two dynamic objects appears to be the optimal choice. However, at $t = 4$ s, the ego-robot is completely surrounded by four obstacles, and it was too late to steer by then. This is the case of RDA, which crashes into the laterally moving obstacle at $t = 5.6$ s. In contrast, with accurate velocity understanding, our DPNet can look ahead into the future, “imagining” that directly passing through is risky, and thus early-executes the left-turn at $t = 2$ s. As such, at $t = 3.6$ s and $t = 5.2$ s, the ego-robot bypasses from behind the laterally moving obstacle. This demonstrates the power of Doppler model-based learning in handling highly-dynamic environments. In terms of other methods, OBCA crashes into the frontally moving obstacle due to its delayed motions caused by low computation frequency. MPC-D-CBF planner makes over-conservative decision upon handling the frontally moving obstacle due to its inaccurate velocity understandings, resulting in a crash eventually.

Quantitative Analysis. To evaluate DPNet in highly-dynamic environments, we adopt DynaBARN [29] benchmark to set up obstacles with randomized trajectories, velocities (6 ± 2 m/s), and accelerations. The following metrics are used: (a) **AvgAcc (m/s²)**: the average acceleration during the execution; (b) **MaxAcc (m/s²)**: the maximum acceleration during the execution; (c) **AvgJerk (m/s³)**: the average jerk during the execution; (d) **IteTime (ms)**: the average iteration time of the solver during the execution; (e) **PassTime (s)**: the average time duration to traverse the area; (f) **PassRate (%)**: the rate of successfully traversing the area without collisions.

Results are reported in Table I, where each obstacle setting is executed 100 times. As the number of obstacles varies from 1 to 7, DPNet consistently outperforms other baselines, with a significantly superior PassRate in dense obstacle settings. In terms of computational efficiency, DPNet achieves a low IteTime, close to that of RDA [8]. However, PassRate of RDA is far below that of DPNet, e.g., degrading by 42.2 % when the number of obstacles is 7. These results confirm that DPNet robustifies collision avoidance with efficient computation. Ab-

¹ Reinforcement learning baselines are not included, since they suffer from a lack of safety assurance in highly-dynamic scenarios.

TABLE I: Quantitative Comparison

Metric	Method				
	OBCA [12]	RDA [8]	MPC-D-CBF [11]	DPNet (Ablation)	DPNet
1 obstacle					
AvgAcc ↓	1.628	1.425	1.725	1.496	1.414
MaxAcc ↓	8.415	8.596	8.506	7.905	7.836
AvgJerk ↓	7.996	7.004	7.671	6.873	6.965
IteTime ↓	147.5	88.0	128.3	99.4	102.1
PassTime ↓	10.362	8.852	9.446	7.976	8.020
PassRate ↑	76.0	87.0	100.0	99.0	100.0
3 obstacles					
AvgAcc ↓	1.743	1.599	2.139	1.537	1.582
MaxAcc ↓	10.085	9.622	9.878	9.561	9.284
AvgJerk ↓	9.251	8.096	10.032	8.101	8.058
IteTime ↓	163.8	114.5	151.7	137.7	135.3
PassTime ↓	12.550	11.376	14.238	10.497	10.194
PassRate ↑	41.0	54.0	74.0	85.0	88.0
5 obstacles					
AvgAcc ↓	1.869	1.606	2.213	1.598	1.571
MaxAcc ↓	9.515	9.481	9.626	9.349	9.577
AvgJerk ↓	9.043	9.128	9.025	8.882	9.134
IteTime ↓	202.8	134.5	194.2	151.3	150.9
PassTime ↓	17.551	15.429	18.345	12.416	11.278
PassRate ↑	21.0	35.0	42.0	55.0	57.0
7 obstacles					
AvgAcc ↓	1.947	1.748	2.114	1.591	1.566
MaxAcc ↓	9.688	9.931	9.786	9.779	9.593
AvgJerk ↓	10.961	11.935	9.601	8.974	9.065
IteTime ↓	234.0	180.9	260.4	218.9	222.4
PassTime ↓	26.710	22.886	28.439	18.054	17.441
PassRate ↑	2.0	26.0	30.0	39.0	45.0

lation study shows that without DT-MPC, DPNet's PassRate drops especially under high obstacle numbers, demonstrating the effectiveness of leveraging real-time Doppler measurements for MPC tuning. Fig. 5 demonstrates how DPNet traverses an area with 5 randomized highly-dynamic obstacles.

Table II presents a sensitivity analysis of DT-MPC under different parameter settings of (α, β) , with 100 runs for each setting. DT-MPC maintains stable performance across various settings. Increasing α or β slightly improves PassRate while degrading AvgJerk, since larger α or β leads to more conservative actions (e.g., abrupt turns) for closer obstacles. Note that DT-MPC achieves a low tuning time $t_{\text{DT-MPC}} (< 1.5 \text{ ms})$, demonstrating its real-time efficiency.

B. Evaluation on Real-World Datasets

To evaluate D-KalmanNet's robustness to real-world sensor noises, we conduct experiments on AevaScene [14] dataset and compare the following methods: **(1) D-KNet**: our proposed D-KalmanNet; **(2) KNet** [5]: the KalmanNet, a physics-informed deep learning method; **(3) KF** [11]: the Kalman filter, a widely-used conventional tracking method; **(4) D-KF** [17]: the conventional Kalman filter aided by Doppler LiDAR.

To quantify tracking accuracy, we use the normalized mean squared error (NMSE) in decibels (dB) following [5]. Specifically, for each vehicle, and at every feasible time step t , we first evaluate the horizon-averaged NMSE over $\mathcal{H}_t$:

$$\text{NMSE}_t = \frac{1}{H} \sum_{h \in \mathcal{H}_t} \frac{(\hat{x}_{h+1|t} - x_{h+1})^2 + (\hat{y}_{h+1|t} - y_{h+1})^2}{x_{h+1}^2 + y_{h+1}^2},$$

TABLE II: Sensitivity Analysis of DT-MPC

Parameter Setting	3 obstacles		5 obstacles		7 obstacles	
	AvgJerk ↓	PassRate ↑	AvgJerk ↓	PassRate ↑	AvgJerk ↓	PassRate ↑
default	8.058	88.0	9.134	57.0	9.056	45.0
$\alpha = 1.0$	8.264	92.0	9.001	59.0	9.077	44.0
$\alpha = 0.5$	8.179	89.0	8.972	54.0	8.796	47.0
$\alpha = 0.1$	7.953	86.0	8.799	48.0	8.436	40.0
$\beta = 1.0$	8.349	90.0	9.206	61.0	9.165	48.0
$\beta = 0.5$	8.370	79.0	8.715	55.0	9.033	39.0
$\beta = 0.1$	8.169	81.0	8.984	57.0	8.912	43.0
$t_{\text{DT-MPC}} (\text{ms})$	0.621		0.997		1.498	

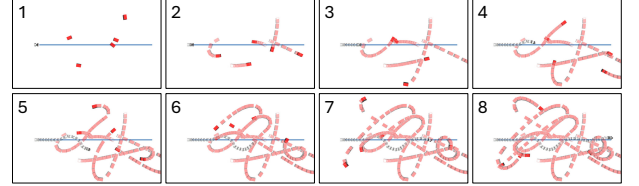


Fig. 5: Visualization of DPNet traversing a highly-dynamic 5-obstacle area, randomized by DynaBARN [29] benchmark. Frames are sampled by 1.25 second interval.

where $(\hat{x}_{h+1|t}, \hat{y}_{h+1|t})$ and (x_{h+1}, y_{h+1}) denote the predicted and ground-truth obstacle positions, respectively. We then convert to the logarithmic scale via $10 \log_{10}(\cdot)$ to obtain a per-step dB-NMSE score. These scores are averaged over vehicle trajectory length to yield a single dB-NMSE value per vehicle, and finally aggregated across all vehicles as **mean $\pm$ std**.

Qualitative Analysis. Fig. 6 demonstrates a visualized result of tracking one rapidly approaching vehicle (at about 25 m/s) on the *Highway* scenario. Among the evaluated methods, D-KNet demonstrates superior performance by effectively leveraging Doppler cues to enable accurate, real-time tracking of rapid objects, achieving the lowest tracking error of -21.18 dB . In contrast, KF and KNet struggle to handle the approaching vehicle with abrupt accelerations. This comparison highlights the importance of integrating model-based learning and Doppler LiDAR measurements for more robust tracking under real-world settings.

Quantitative Analysis. Table III reports the results across tracking frequencies (2-10 Hz), prediction horizons ($\mathcal{H}_t = 5, 10$), and driving scenarios (*Highway*, *City*). Across all settings, D-KNet consistently achieves the lowest NMSE and significantly outperforms all baselines. Notably, at $\mathcal{H}_t = 5$, D-KNet attains $-45.00 \pm 7.35 \text{ dB}$ in the *City* scenario at 10 Hz and surpasses the second best D-KF [17] by 12.26 dB. Even at $\mathcal{H}_t = 10$, D-KNet maintains robustness to long-horizon uncertainty, whereas other methods suffer considerable degradation. Tighter standard deviations of D-KNet further confirm its stability across diverse obstacle dynamics. These results demonstrate the superior reliability of D-KNet, especially under real-world constraints such as varying sensor noise, limited processing frequency, and scenario complexity.

C. More Quantitative Results

We also deploy D-KNet on an NVIDIA Jetson Orin NX 16 GB to evaluate its real-time tracking efficiency on a resource-constrained platform. It is found that D-KNet consumes only about 107 MB of GPU memory. As Table IV presents, it achieves inference rates of over 100 Hz when tracking a single fast-moving obstacle. Notably, even with

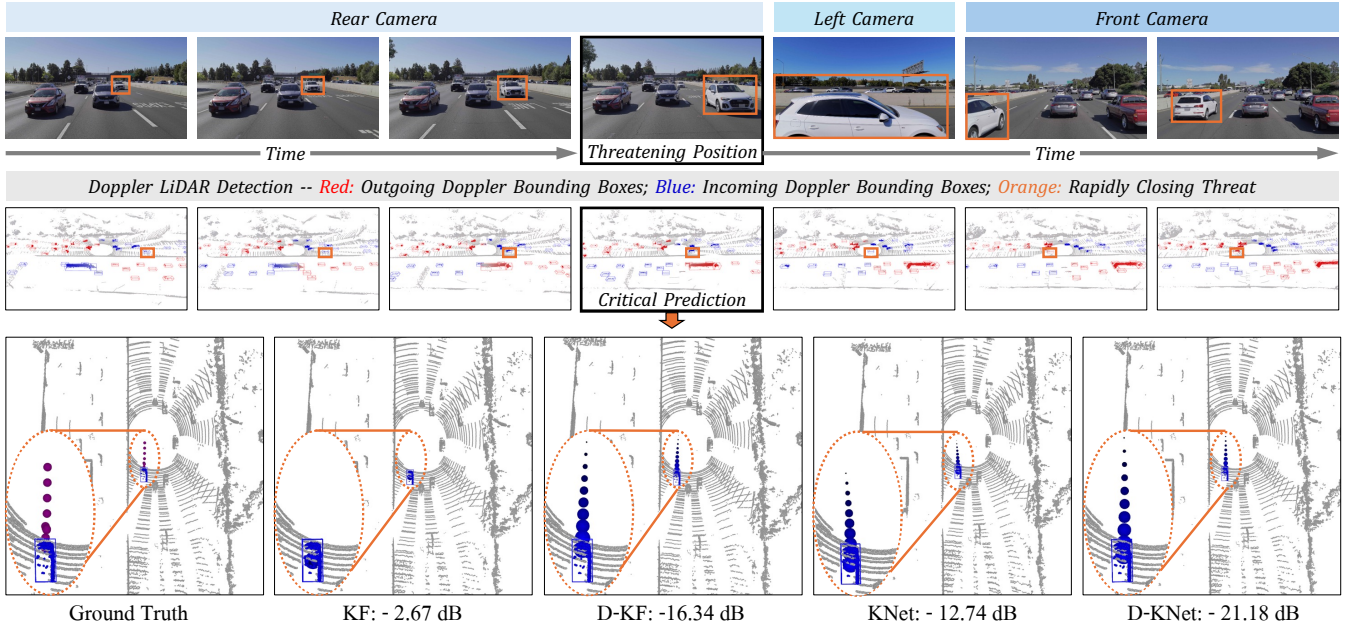


Fig. 6: Visualization of critical predictions for a rapidly approaching vehicle (at about 25 m/s, highlighted in orange) in *Highway* scenario of AevaScene [14]. The upper half shows Doppler LiDAR’s environmental dynamics detection (outgoing/incoming bounding boxes marked in red/blue). The lower half provides detailed comparison of trajectory predictions for the threat.

TABLE III: Prediction NMSE (dB) by Mean $\pm$ Std

$\mathcal{H}_t$	Method	Tracking Frequency		
		10 Hz	5 Hz	2 Hz
5	-	<i>Highway scenario</i>		
	KF [11]	-22.66 $\pm$ 6.41	-16.62 $\pm$ 5.24	-7.18 $\pm$ 3.84
	KNet [5]	-23.38 $\pm$ 7.82	-17.79 $\pm$ 7.77	-12.56 $\pm$ 7.16
	D-KF [17]	-27.15 $\pm$ 6.39	-21.66 $\pm$ 4.77	-14.09 $\pm$ 3.60
	D-KNet	-35.80 $\pm$ 8.62	-29.11 $\pm$ 7.17	-20.22 $\pm$ 2.45
	-	<i>City scenario</i>		
	KF [11]	-27.15 $\pm$ 6.35	-20.48 $\pm$ 4.79	-9.15 $\pm$ 6.64
	KNet [5]	-29.26 $\pm$ 5.47	-23.66 $\pm$ 5.98	-17.38 $\pm$ 7.36
	D-KF [17]	-32.74 $\pm$ 6.18	-27.60 $\pm$ 5.11	-19.20 $\pm$ 5.03
	D-KNet	-45.00 $\pm$ 7.35	-37.67 $\pm$ 6.11	-29.63 $\pm$ 6.76
10	-	<i>Highway scenario</i>		
	KF [11]	-18.50 $\pm$ 6.53	-11.49 $\pm$ 5.92	1.88 $\pm$ 6.47
	KNet [5]	-17.50 $\pm$ 7.49	-11.83 $\pm$ 7.64	-6.32 $\pm$ 6.73
	D-KF [17]	-23.68 $\pm$ 6.44	-17.36 $\pm$ 4.65	-6.61 $\pm$ 4.92
	D-KNet	-26.28 $\pm$ 8.33	-18.88 $\pm$ 6.60	-8.56 $\pm$ 4.66
	-	<i>City scenario</i>		
	KF [11]	-22.55 $\pm$ 6.46	-14.14 $\pm$ 5.27	8.05 $\pm$ 0.30
	KNet [5]	-23.19 $\pm$ 5.54	-17.34 $\pm$ 6.12	-9.81 $\pm$ 8.92
	D-KF [17]	-29.67 $\pm$ 6.90	-22.97 $\pm$ 5.59	-10.59 $\pm$ 7.26
	D-KNet	-34.30 $\pm$ 7.65	-25.96 $\pm$ 6.82	-16.41 $\pm$ 9.47

10 simultaneously tracked obstacles, D-KNet sustains about 15 Hz with low CPU usage of 42.0 %. These confirm D-KNet’s suitability for lightweight deployment on robot platforms.

Table V shows per-step NMSE (dB) by **mean $\pm$ std** over a 10-step prediction horizon on the AevaScenes [14] dataset. D-KNet consistently outperforms all baselines at every step, with margin widening especially in short steps (e.g., -48.81 dB of D-KNet and -16.49 dB of KF for step 1 in *City*). The KF and D-KF methods exhibit nearly flat error growth, indicating trivial estimates under transition model mismatch. However, D-KNet and KNet show physically reasonable error

TABLE IV: Hardware Efficiency of D-KNet

Obstacle Number	1	2	3	4	5	8	10
Execution Time (s)	0.008	0.016	0.025	0.032	0.037	0.050	0.070
Frequency (Hz)	126.57	62.08	39.93	31.53	27.10	19.87	14.36
CPU Usage (%)	5.0	8.0	27.0	38.0	40.0	41.0	42.0

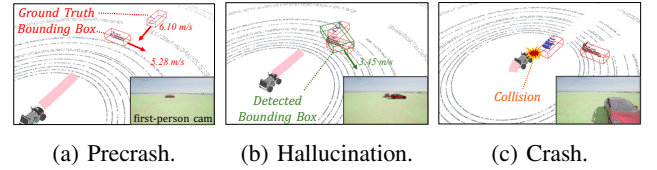


Fig. 7: Failure mode analysis.

accumulation, demonstrating the effectiveness of model-based learning against dynamics uncertainty. Notably, D-KNet maintains tighter standard deviations and significantly lower mean error, verifying that the Doppler LiDAR integration enhances both accuracy and robustness for long-horizon predictions.

D. Limitation and Future Work

In practice, bounding box detection noises (e.g., jitters, false positives) may affect obstacle linear velocity estimation and then downstream planning. Fig. 7 demonstrates a failure mode case where a severe box under-segmentation occurs when two rapid obstacles (moving at 6.10 m/s and 5.28 m/s) approach, as depicted in (b). This contaminates linear velocity estimation and obstacle tracking with a hallucinated object moving at 3.45 m/s and propagates to downstream planning, resulting in a collision failure eventually, shown in (c). To address this, our future work aims to develop bounding-box-free methods for Doppler LiDAR obstacle tracking. In terms of controller parameter tuning, DT-MPC explores Doppler-guided tuning with a heuristic strategy, which is in general suboptimal for minimizing $\mathcal{L}$, though proved to be efficient and effective.

TABLE V: Per-horizon-step Prediction NMSE (dB) by Mean $\pm$ Std

Method	1	2	3	4	5	6	7	8	9	10
Highway scenario										
KF [11]	-15.53 $\pm$ 8.80	-15.15 $\pm$ 9.63	-14.72 $\pm$ 8.45	-14.24 $\pm$ 8.26	-13.73 $\pm$ 8.09	-13.20 $\pm$ 7.90	-12.66 $\pm$ 7.72	-12.08 $\pm$ 7.53	-11.52 $\pm$ 7.33	-10.96 $\pm$ 7.15
KNet [5]	-30.82 $\pm$ 3.80	-24.80 $\pm$ 4.05	-21.44 $\pm$ 4.28	-18.93 $\pm$ 4.50	-16.93 $\pm$ 4.70	-15.24 $\pm$ 4.87	-13.78 $\pm$ 5.03	-12.42 $\pm$ 5.19	-11.23 $\pm$ 5.34	-10.16 $\pm$ 5.50
D-KF [17]	-16.10 $\pm$ 10.37	-16.01 $\pm$ 9.51	-15.88 $\pm$ 9.19	-15.74 $\pm$ 8.91	-15.56 $\pm$ 8.64	-15.36 $\pm$ 8.47	-15.13 $\pm$ 8.33	-14.88 $\pm$ 8.23	-14.58 $\pm$ 8.14	-14.27 $\pm$ 8.10
D-KNet	-41.39 $\pm$ 3.23	-34.79 $\pm$ 3.34	-30.17 $\pm$ 3.47	-26.63 $\pm$ 3.58	-23.74 $\pm$ 3.73	-21.27 $\pm$ 3.90	-19.14 $\pm$ 4.10	-17.23 $\pm$ 4.29	-15.45 $\pm$ 4.52	-13.89 $\pm$ 4.74
City scenario										
KF [11]	-16.49 $\pm$ 7.12	-16.34 $\pm$ 6.83	-16.32 $\pm$ 6.69	-16.15 $\pm$ 6.54	-15.92 $\pm$ 6.42	-15.65 $\pm$ 6.30	-15.34 $\pm$ 6.17	-14.98 $\pm$ 6.04	-14.59 $\pm$ 5.91	-14.16 $\pm$ 5.77
KNet [5]	-38.22 $\pm$ 3.05	-32.42 $\pm$ 3.10	-28.90 $\pm$ 3.16	-26.26 $\pm$ 3.42	-24.13 $\pm$ 3.57	-22.3 $\pm$ 3.72	-20.68 $\pm$ 3.88	-19.22 $\pm$ 3.96	-17.87 $\pm$ 3.95	-16.59 $\pm$ 4.06
D-KF [17]	-16.29 $\pm$ 8.89	-16.26 $\pm$ 8.19	-16.23 $\pm$ 7.78	-16.19 $\pm$ 7.47	-16.17 $\pm$ 7.26	-16.13 $\pm$ 7.07	-16.13 $\pm$ 6.88	-16.13 $\pm$ 6.68	-16.12 $\pm$ 6.51	-16.11 $\pm$ 6.34
D-KNet	-48.81 $\pm$ 3.01	-41.47 $\pm$ 3.09	-36.48 $\pm$ 3.21	-32.63 $\pm$ 3.25	-29.44 $\pm$ 3.17	-27.76 $\pm$ 3.31	-24.45 $\pm$ 3.57	-22.41 $\pm$ 3.62	-20.59 $\pm$ 3.65	-18.95 $\pm$ 3.76

To address this, our future work aims to integrate gradient-informed techniques like differentiable optimization or policy search [30], [31] for optimal Doppler-guided tuning, while maintaining model efficiency.

VI. CONCLUSION

This paper has introduced DPNet, a Doppler LiDAR model-based learning method for collision avoidance among rapid-motion obstacles. DPNet achieves both high frequency and accuracy in tracking and planning. Compared to benchmark schemes, it reduces the navigation time by 6%–30%, increases the success rate by up to 16%, and reduces the tracking error by over 10 dB. It was demonstrated that DPNet adapts to platforms with limited computational resources. Ablation studies confirmed the indispensability of incorporating Doppler information into controller tuning and motion planning.

REFERENCES

- [1] Z. Han, M. Tian, Z. Gongye, D. Xue, J. Xing, Q. Wang, Y. Gao, J. Wang, C. Xu, and F. Gao, “Hierarchically depicting vehicle trajectory with stability in complex environments,” *Sci. Robot.*, vol. 10, no. 103, p. eads4551, 2025.
- [2] C. Flores, P. Merdrignac, R. de Charette, F. Navas, V. Milanés, and F. Nashashibi, “A cooperative car-following/emergency braking system with prediction-based pedestrian avoidance capabilities,” *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 5, pp. 1837–1846, 2018.
- [3] A. Milan, S. H. Rezatofighi, A. Dick, I. Reid, and K. Schindler, “Online multi-target tracking using recurrent neural networks,” in *Proc. AAAI*, vol. 31, no. 1, 2017, pp. 4225–4232.
- [4] V. Lefkopoulou, M. Menner, A. Domahidi, and M. N. Zeilinger, “Interaction-aware motion prediction for autonomous driving: A multiple model kalman filtering scheme,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 1, pp. 80–87, 2020.
- [5] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. Van Sloun, and Y. C. Eldar, “Kalmannet: Neural network aided kalman filtering for partially known dynamics,” *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.
- [6] H. Liao, C. Wang, Z. Li, Y. Li, B. Wang, G. Li, C. Xu *et al.*, “Physics-informed trajectory prediction for autonomous driving under missing observation,” in *IJCAI*, 2024, pp. 6841–6849.
- [7] B. Hessel, H. Vhavle, and Y. Chen, “Dicp: Doppler iterative closest point algorithm,” *arXiv preprint arXiv:2201.11944*, 2022.
- [8] R. Han, S. Wang, S. Wang, Z. Zhang, Q. Zhang, Y. C. Eldar, Q. Hao, and J. Pan, “Rda: An accelerated collision free motion planner for autonomous navigation in cluttered environments,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 3, pp. 1715–1722, 2023.
- [9] Y. Zhang, G. Tian, L. Wen, X. Yao, L. Zhang, Z. Bing, W. He, and A. Knoll, “Online efficient safety-critical control for mobile robots in unknown dynamic multi-obstacle environments,” in *Proc. IROS*, IEEE, 2024, pp. 12 370–12 377.
- [10] R. Tao, S. Cheng, X. Wang, S. Wang, and N. Hovakimyan, “Diffunet: Closed-loop learning for model predictive control,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 8, pp. 7294–7301, 2024.
- [11] Z. Jian, Z. Yan, X. Lei, Z. Lu, B. Lan, X. Wang, and B. Liang, “Dynamic control barrier function-based model predictive control to safety-critical obstacle-avoidance of mobile robot,” in *Proc. ICRA*, 2023, pp. 3679–3685.
- [12] X. Zhang, A. Liniger, and F. Borrelli, “Optimization-based collision avoidance,” *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 3, pp. 972–983, 2020.
- [13] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “Carla: An open urban driving simulator,” in *Proc. CoRL*, 2017, pp. 1–16.
- [14] G. N. Narasimhan, H. Vhavle, K. B. Vishvanatha, and J. Reuther, “Aevascenes: A dataset and benchmark for fmcw lidar perception,” 2025. [Online]. Available: <https://scenes.aeva.com/>
- [15] Y. Shi, K. Jiang, X. Zhao, K. Qian, C. Xie, T. Wen, M. Yang, and D. Yang, “Pod: Predictive object detection with single-frame fmcw lidar point cloud,” *arXiv preprint arXiv:2504.05649*, 2025.
- [16] Y. Gu, H. Cheng, K. Wang, D. Dou, C. Xu, and H. Kong, “Learning moving-object tracking with fmcw lidar,” in *Proc. IROS*, 2022, pp. 3747–3753.
- [17] X. Peng and J. Shan, “Detection and tracking of pedestrians using doppler lidar,” *Remote Sens.*, vol. 13, no. 15, p. 2952, 2021.
- [18] Y. Wu, D. J. Yoon, K. Burnett, S. Kammel, Y. Chen, H. Vhavle, and T. D. Barfoot, “Picking up speed: Continuous-time lidar-only odometry using doppler velocity measurements,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 1, pp. 264–271, 2022.
- [19] M. Zhao, J. Wang, T. Gao, C. Xu, and H. Kong, “FMCW-LIO: A doppler lidar-inertial odometry,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5727–5734, 2024.
- [20] D. J. Yoon, K. Burnett, J. Laconte, Y. Chen, H. Vhavle, S. Kammel, J. Reuther, and T. D. Barfoot, “Need for speed: Fast correspondence-free lidar-inertial odometry using doppler velocity,” in *Proc. IROS*, 2023, pp. 5304–5310.
- [21] M. Zhao, J. Wang, T. Gao, C. Xu, and H. Kong, “Free-init: Scan-free, motion-free, and correspondence-free initialization for doppler lidar-inertial systems,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 12, pp. 11 329–11 336, 2024.
- [22] Aeva-Inc., “Carla-Aeva,” <https://github.com/aevainc/carla-aeva>, 2024.
- [23] J. Zhou, Y. Gao, O. Johansson, B. Olofsson, and E. Frisk, “Robust predictive motion planning by learning obstacle uncertainty,” *IEEE Trans. Control Syst. Technol.*, vol. 33, no. 3, pp. 1006–1020, 2025.
- [24] R. Zhang, C. Yu, J. Chen, C. Fan, and S. Gao, “Learning-based motion planning in dynamic environments using gnns and temporal encoding,” in *Proc. NeurIPS*, vol. 35, 2022, pp. 30 003–30 015.
- [25] Y. Shi and K. Zhang, “Advanced model predictive control framework for autonomous intelligent mechatronic systems: A tutorial overview and perspectives,” *Annu. Rev. Control*, vol. 52, pp. 170–196, 2021.
- [26] R. Han, S. Wang, S. Wang, Z. Zhang, J. Chen, S. Lin, C. Li, C. Xu, Y. C. Eldar, Q. Hao *et al.*, “Neupan: Direct point robot navigation with end-to-end model-based learning,” *IEEE Trans. Robot.*, vol. 41, pp. 2804–2824, 2025.
- [27] S. Zhang, H. Li, S. Zhang, S. Wang, D. W. K. Ng, and C. Xu, “Multi-uncertainty aware autonomous cooperative planning,” in *Proc. IROS*, 2024, pp. 1018–1025.
- [28] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, “Motion planning with sequential convex optimization and convex collision checking,” *Int. J. Rob. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [29] A. Nair, F. Jiang, K. Hou, Z. Xu, S. Li, X. Xiao, and P. Stone, “Dynabarn: Benchmarking metric ground navigation in dynamic environments,” in *Proc. SSRR*, 2022, pp. 347–352.
- [30] S. Cheng, M. Kim, L. Song, C. Yang, Y. Jin, S. Wang, and N. Hovakimyan, “Diffune: Auto-tuning through auto-differentiation,” *IEEE Trans. Robot.*, vol. 40, pp. 4085–4101, 2024.
- [31] Y. Song and D. Scaramuzza, “Policy search for model predictive control with application to agile drone flight,” *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2114–2130, 2022.